17 GUARDS — PROTECTING ROUTES IN NESTJS

A Guard is a class that decides ONE thing: is this request ALLOWED to reach the route handler at all? It runs BEFORE the handler (and before pipes), and either lets the request through or blocks it outright — typically for authentication ("who are you?") and authorization ("are you allowed to do this?").

```
Client Request
     |
Middleware
     |
GUARDS  <-- runs here. Returns true -> continue. false/throws -> blocked.
     |
Interceptors (before)
     |
Pipes            (see theory/09-pipes.txt)
     |
Route Handler
     |
Interceptors (after)
     |
Response
```

If a guard blocks the request, the handler NEVER RUNS — no pipes, no service calls, nothing. This is different from a pipe: a pipe assumes the request is allowed and just checks/shapes its DATA; a guard decides whether the request is allowed to happen AT ALL.

THE CanActivate INTERFACE

Every guard implements CanActivate, with one method: canActivate.

```
  import { CanActivate, ExecutionContext, Injectable } from '@nestjs/common';

  @Injectable()
  export class MyGuard implements CanActivate {
    canActivate(context: ExecutionContext): boolean {
      // return true  -> request proceeds
      // return false -> Nest throws a 403 Forbidden automatically
      // OR throw your own exception (e.g. UnauthorizedException) for
      //    a more specific error message/status code
      return true;
    }
  }

context (ExecutionContext)   Gives access to the underlying request.
                              For a normal HTTP route:
                                const req = context.switchToHttp().getRequest();
                              (Nest also supports WebSocket/RPC
                              contexts — switchToHttp() is the one
                              you'll use for a REST API like this one.)
```

canActivate can also return a Promise<boolean> or an Observable<boolean> — useful when the check needs to await something (e.g. verifying a token against a database or an external auth service).

A REAL, RUNNABLE EXAMPLE — API KEY GUARD

A simple guard that requires an `x-api-key` header matching a known value — no real user accounts, just a shared secret. Good first guard to understand the mechanism before adding real auth (JWT, sessions).

```
// src/common/guards/api-key.guard.ts
import {
  CanActivate,
  ExecutionContext,
  Injectable,
  UnauthorizedException,
} from '@nestjs/common';
import { Request } from 'express';

const VALID_API_KEY = 'secret123'; // in a real app: read from ConfigService/env

@Injectable()
export class ApiKeyGuard implements CanActivate {
  canActivate(context: ExecutionContext): boolean {
    const request = context.switchToHttp().getRequest<Request>();
    const key = request.headers['x-api-key'];

    if (key !== VALID_API_KEY) {
      throw new UnauthorizedException('Invalid or missing API key');
    }

    return true;
  }
}
```

Applied to protect deleting a task (the most "destructive" route in this project's task module):

```
// task.controller.ts
@UseGuards(ApiKeyGuard)
@Delete(':id')
remove(@Param('id', ParsePositiveIntPipe) id: number): Task {
  return this.taskService.remove(id);
}
```

Request without the header (or the wrong key):

```
DELETE /task/1
-> 401 { "message": "Invalid or missing API key", ... }
-> remove() NEVER RUNS — TaskService.remove is never called.
```

Request with the right header:

```
DELETE /task/1  (header: x-api-key: secret123)
-> guard returns true -> handler runs normally.
```

WHERE @UseGuards() CAN BE APPLIED

Same three scopes pipes support, plus a true global option:

1. Method-level (protect ONE route): @UseGuards(ApiKeyGuard) @Delete(':id') remove(...) { ... }

2. Controller-level (protect EVERY route in the controller): @UseGuards(ApiKeyGuard) @Controller('task') export class TaskController { ... }

3. Global, in main.ts (protect the ENTIRE app): app.useGlobalGuards(new ApiKeyGuard()); (Awkward if the guard needs DI — same caveat as global custom pipes, see theory/11-builtin-vs-custom-pipes.txt.)

4. Global via a provider token (DI-friendly global option): // app.module.ts import { APP_GUARD } from '@nestjs/core';

```
  @Module({
    providers: [
      { provide: APP_GUARD, useClass: ApiKeyGuard },
    ],
  })
  export class AppModule {}

This is the preferred way to apply a guard app-wide when it needs
injected dependencies — Nest resolves it through the normal DI
container instead of you constructing it by hand.
```

COMBINING GUARDS WITH ROUTE-SPECIFIC METADATA (@SetMetadata / custom decorator)

Real apps rarely hardcode one rule per guard — instead, a route is tagged with metadata, and ONE guard reads that metadata to decide.

Common pattern for role-based access:

```
// roles.decorator.ts
import { SetMetadata } from '@nestjs/common';
export const Roles = (...roles: string[]) => SetMetadata('roles', roles);

// roles.guard.ts
@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    const requiredRoles = this.reflector.get<string[]>(
      'roles',
      context.getHandler(),
    );
    if (!requiredRoles) return true; // route has no @Roles() -> unrestricted

    const { user } = context.switchToHttp().getRequest();
    return requiredRoles.includes(user?.role);
  }
}

// usage
@Roles('admin')
@UseGuards(RolesGuard)
@Delete(':id')
remove(...) { ... }
```

This is the standard shape of Nest's official auth/RBAC pattern — one generic guard, reused everywhere, driven by per-route metadata instead of writing a new guard per rule.

GUARD vs PIPE vs INTERCEPTOR — QUICK DISTINCTION (RECAP)

Guard	"Is this request ALLOWED to happen at all?" Runs FIRST. Blocks entirely (401/403) if it fails. Scope: authentication / authorization.
Pipe	"Is this ARGUMENT'S data valid/correctly shaped?" Runs after guards, right before the handler. Scope: input transformation/validation (see theory/09-pipes.txt, theory/13-dto.txt).
Interceptor	Wraps the whole call, runs logic before AND after the handler (logging, response transforming, timeouts). Scope: cross-cutting behavior around the request.

A guard failing means the handler (and any pipes on its arguments) never run at all — it's the earliest point at which a request can be rejected in Nest's pipeline.

★ RULE OF THUMB

Reach for a guard the moment a route needs to ask "who is calling this, and are they allowed to?" — not "is the data they sent well-formed" (that's a pipe's job). Start with one broad guard per concern (an ApiKeyGuard, then later a JwtAuthGuard) and layer route-specific rules on top via metadata + a single reusable guard (like RolesGuard above), rather than writing a new guard class for every individual rule.

NOTE ApiKeyGuard above is a DESIGNED example for teaching purposes —
it has not been added to src/common/ yet. Ask if you'd like it wired into the real task.controller.ts to test against, the same way ParsePositiveIntPipe was.

See also: theory/09-pipes.txt, theory/10-custom-pipes.txt, theory/06-decorators.txt, theory/07-dependency-injection.txt